TaskAnalyzer

Documento de Arquitetura do Sistema (System Design Document)

Projeto: TaskAnalyzer | Disciplina: Bootcamp III - Fase 3
Aluno: João Victor Pereira Alves | Turma: A | Ano: 2026

1. Objetivo e escopo

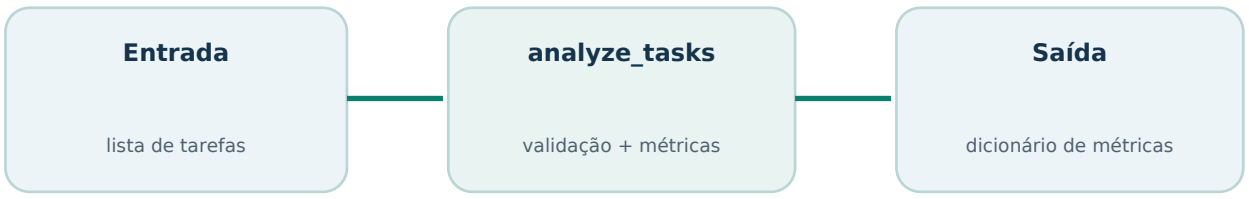
O TaskAnalyzer analisa uma lista de tarefas fornecida em memória, valida seus campos e retorna métricas de produtividade. O projeto aplica Spec-Driven Development (SDD), testes automatizados, integração contínua e execução isolada por Docker. Não existe interface gráfica, serviço HTTP nem persistência de dados no escopo implementado.

2. Requisitos de referência

As fontes de verdade do projeto são CONTEXT_RULES.md, specs/task_analyzer_spec.md e a implementação em src/task_analyzer.py. O arquivo specs/requirements.yaml organiza os requisitos para consulta. A análise respeita os valores de prioridade baixa, média, alta e os status pendente, concluída, cancelada.

3. Visão arquitetural

O sistema é uma função Python chamada analyze_tasks(tasks). A automação é organizada ao redor do núcleo funcional, sem alterar seu contrato público.



pytest • GitHub Actions • Docker Compose

4. Dados de entrada e saída

Entrada: lista de dicionários contendo id, titulo, prioridade, data_criacao, prazo e status; data_conclusao é opcional, porém obrigatória quando o status é concluída. Datas devem ser objetos datetime. IDs devem ser únicos.

Saída: dicionário com quantidade_analisada, quantidade_concluida, quantidade_atrasada, tempo_medio_conclusao_horas, taxa_atraso_percentual e por_prioridade (totais e contagens por status/atraso).

5. Regras de cálculo

Indicador	Regra implementada
Tempo médio de conclusão	Média das horas entre criação e conclusão apenas para tarefas concluídas.
Atraso	Concluída com data de conclusão posterior ao prazo.
Taxa de atraso	(Quantidade de concluídas com atraso / quantidade de concluídas) × 100.
Sem concluídas	Tempo médio e taxa de atraso retornam 0.0.
Por prioridade	Contagem de total, concluídas, pendentes, canceladas e atrasadas.

6. Validações e falhas

A função verifica tipo da entrada, formato de cada tarefa, presença dos campos obrigatórios, IDs repetidos, campos textuais, prioridade e status válidos, coerência das datas e relação entre status e data de conclusão. Entradas inválidas produzem TaskValidationError com mensagem descritiva. O módulo logging registra o início e o encerramento de análises concluídas.

7. Componentes do repositório

src/task_analyzer.py — Implementação da análise e validações

tests/test_harness.py — 17 testes automatizados em pytest

.github/workflows/ci.yml — Execução automatizada de testes em GitHub Actions

Dockerfile — Imagem Linux baseada em python:3.11-slim

docker-compose.yml — Orquestração do contêiner de testes

requirements.txt — Dependência pytest

CONTEXT_RULES.md — Padrões, limites e governança para uso de IA

specs/ — Especificações e documentação de arquitetura

8. Fluxo de execução e verificação

1. O chamador fornece uma lista de tarefas à função analyze_tasks.
2. A função valida os dados e percorre os registros, acumulando contagens e durações.
3. Os resultados são calculados e retornados em estrutura de dicionário.
4. python -m pytest -v valida os cenários locais.
5. docker compose up --build --abort-on-container-exit --exit-code-from taskanalyzer constrói o ambiente e executa a suíte isoladamente.
6. O GitHub Actions executa o workflow de CI após alterações no repositório.

9. Evidências verificadas

Na validação apresentada pelo desenvolvedor, a suíte local aprovou 17/17 testes. A execução em contêiner Linux com Python 3.11.17 e pytest 9.1.1 também aprovou 17/17 testes, com código de saída 0. A última execução apresentada do GitHub Actions, correspondente ao commit de conteinerização 7ff58a5, aparece como aprovada. Essas evidências não equivalem a garantia de ausência de defeitos em cenários ainda não testados.

10. Decisões, limites e próximos cuidados

A implementação utiliza biblioteca padrão do Python e pytest como dependência de testes. Não armazena dados em arquivos ou bancos. A porcentagem de atraso usa tarefas concluídas como denominador, interpretação adotada na implementação e formalizada no YAML; recomenda-se manter a especificação Markdown igualmente explícita. O sistema não inclui deploy de serviço ou API: o contêiner foi desenhado para executar testes. Novas alterações devem seguir o contrato SDD, receber testes e ser validadas por revisão humana.

11. Referências do projeto

Repositório: github.com/palvesjoao7-prog/sdd-task-analyzer
Artefatos: CONTEXT_RULES.md, specs/task_analyzer_spec.md, specs/requirements.yaml, tests/test_harness.py, src/task_analyzer.py, Dockerfile e .github/workflows/ci.yml.